

VelaDial

Project Overview

VelaDial

A quiet, local-first bedroom lighting controller built with Home Assistant, ESPHome, a round rotary display, and a bedside gesture sensor.

VelaDial controls a group of Tuya RGB bulbs without relying on voice commands, cloud latency, relay clicks, or a phone screen at night.

Hardware concept

- Door-side ELECROW 1.28 in rotary touch display for power, brightness, and color presets.
- Bedside ESP32-C6 with APDS-9960 gesture sensor for silent hand-wave control.
- Home Assistant as the local hub.
- LocalTuya or an equivalent local LAN integration for the existing Tuya bulbs.

Sensor expansion

VelaDial includes additional sensors for adaptive behavior and improved reliability:

NODE	SENSOR	I2C ADDRESS	PURPOSE
Door-side ESP32-S3	TSL2591	0x29	Adaptive display brightness based on ambient light
Door-side ESP32-S3	SHT45	0x44	Secondary temperature/humidity diagnostics
Bedside ESP32-C6	APDS-9960	0x39	Directional gestures (left/right) for light on/off
Bedside ESP32-C6	VL53L4CD	0x29	Deliberate hand-hold detection for nightlight mode

The door-side and bedside nodes use separate I2C buses. TSL2591 and VL53L4CD both use 0x29, but this is not a conflict because they are on different ESP32 devices. No I2C multiplexer is required for the first build.

Key sensor roles

- TSL2591 controls adaptive display brightness only. It does not control room bulbs directly.

- SHT45 provides secondary diagnostics only. It is not a required main page element.
- VL53L4CD enables deliberate hand-hold nightlight behavior (stable hand at 5–10 cm for ~1.5 s).
- APDS-9960 remains the directional gesture sensor for light on/off.

Main UI

The door-side display has exactly 3 primary control pages, plus a hidden device-level theme selector:

1. Power
2. Brightness
3. Presets
4. Theme Selector (hidden, accessed via long-press >1.5s)

The firmware includes **20 selectable UI themes** that can be switched on-device without reflashing. Temperature/humidity data is secondary and does not appear on the main pages.

Repository layout

```
.  
├── PROJECT_CONTEXT_FOR_AI.md  
├── AGENTS.md  
├── docs/  
├── esphome/  
└── hardware/
```

Start here

For a coding assistant, paste or open `PROJECT_CONTEXT_FOR_AI.md` first. It contains the hardware selection, pinout, sensor architecture, UI requirements, and next implementation target.

For human review, read in this order:

1. `PROJECT_CONTEXT_FOR_AI.md`
2. `docs/01_PRD.md`
3. `docs/02_TRD.md`
4. `docs/03_App_Flow.md`
5. `docs/04_UI_UX_Design_Brief.md`
6. `docs/05_Backend_Schema.md`
7. `docs/06_Implementation_Plan.md`

8. `docs/07_Research_Alternatives.md`
9. `esphome/`
10. `hardware/`

Current firmware status

- `esphome/door_side_rotary.yaml` — **Compile-passing multi-theme firmware candidate** with 20 selectable UI themes. Compile PASSED (ESPHome 2026.5.0, ESP-IDF 5.5.4, 0 errors). RAM: 17.5%, Flash: 65.5%. Hardware validation pending. Not physically validated. Further visual refinement may follow hardware testing.
- `esphome/bedside_gesture.yaml` — ESP32-C6 + APDS-9960 gesture controller. Hardware validation NOT YET TESTED.

The next major work items are hardware validation:

1. Validate the ELECROW board revision and pinout against the actual hardware.
2. Verify I2C scans on both nodes (door-side: `0x29`, `0x44`; bedside: `0x39`, `0x29`).
3. Flash the door-side firmware candidate and verify all 20 themes render correctly on the physical display.
4. Verify the VL53L4CD ESPHome support path before writing hold-nightlight firmware. If blocked, surface the decision to the owner before any fallback.
5. Bedside bring-up: APDS-9960 standalone first; VL53L4CD support verification second. **Do not implement sensor fusion in v1.**

v1 bedside behavior is APDS-9960 standalone left/right gestures plus VL53L4CD standalone hand-hold nightlight (only if VL53L4CD support is verified). VL53L4CD/APDS-9960 sensor fusion is v2 / future only — not a first-build requirement.

Design principles

- Silent by default.
- Local-first control path.
- Low light pollution for bedroom use.
- Small, reviewable firmware and documentation changes.
- Useful for AI-assisted development, but still human-reviewed.

Private configuration

Keep live WiFi credentials, API credentials, Tuya local keys, Home Assistant tokens, and generated ESPHome build output out of the repository. Store private values only in your local ESPHome/Home Assistant configuration.